

Chapter 1 : The Intel 8086 Microprocessor

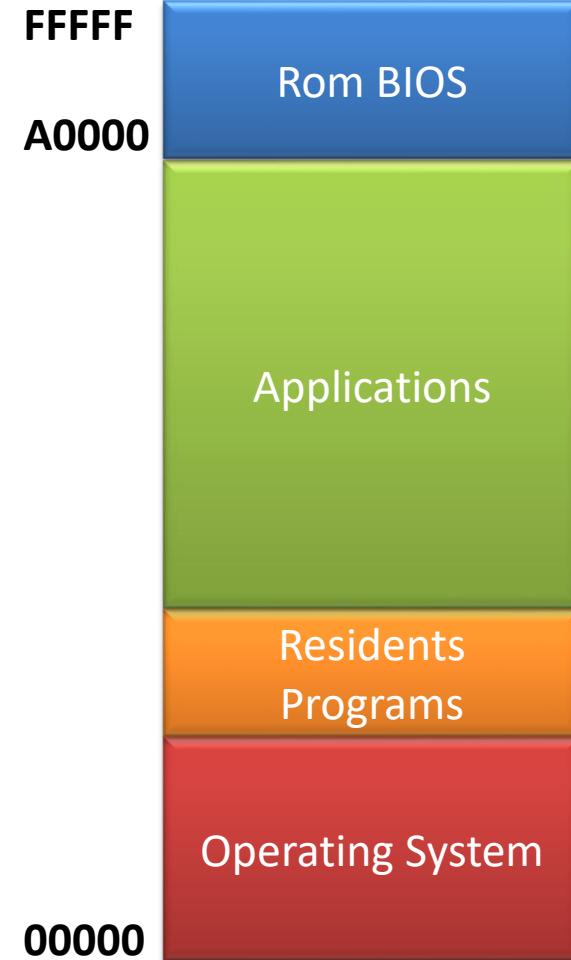
Next

- Why study The Intel 8086 Microprocessor
- The 8086's architecture
- The 8086's/8088's registers
- The 8086's memory management
- The 8086's Assembly Program Structure
- The 8086's addressing mode

Memory Organization

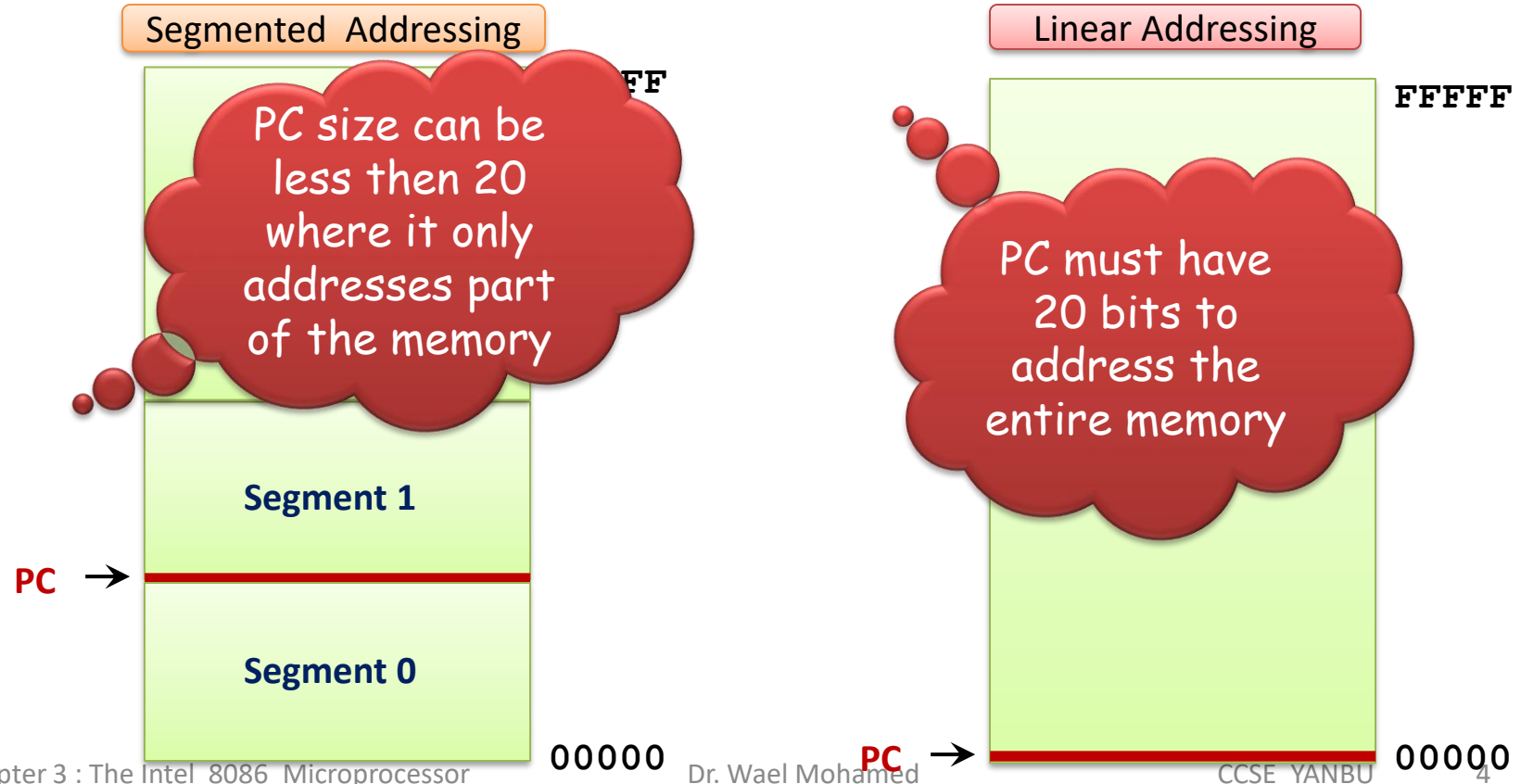
❑ The 1MB memory of the 8086 is organized as follows:

- The operating system is located starting from address 00000
- After OS, the Residents programs installed by the OS is located
- The last area of the memory from address A0000 to FFFFFF is reserved for the ROM BIOS
- ROM BIOS usually contains the startup program that enable the computer to boot-up probably
- The remaining area is free for user's applications



Memory Segmentation

- ❑ Two types of memory organizations are commonly used
 - *Linear Addressing*
 - *Segmented Addressing*
- ❑ In *Linear addressing* the whole memory is available as one linear array
- ❑ In *Segmented addressing*, memory space is divided into segments where only one segment can be accessed at a time

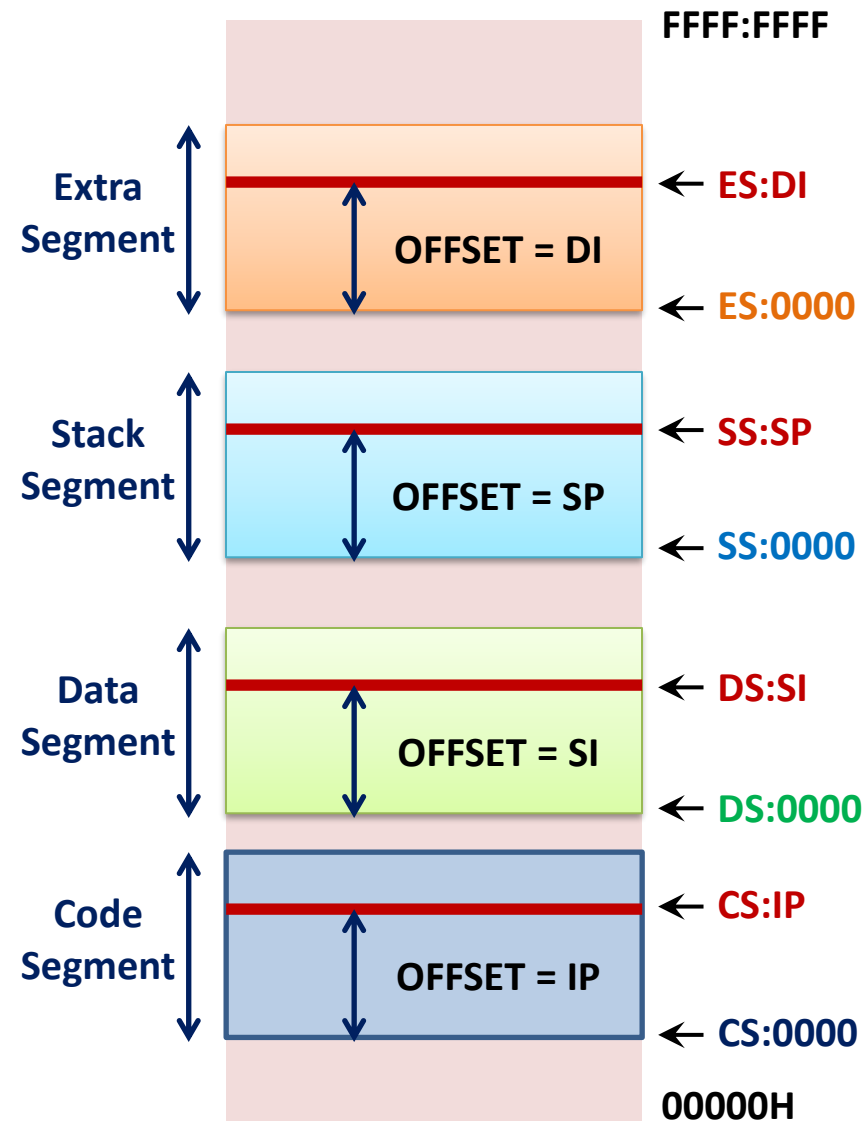


8086 Memory Segmentation

- ❑ The total memory “1 MB” is divided into 64 kb segments
- ❑ Each segment is addressed by one of the *segment registers*
- ❑ The 16-bit contents of the segment register gives the *starting/base* address of a particular segment
- ❑ Another part called *offset*-address is used to address a byte within the selected segment
- ❑ The offset address is also 16-bit wide, so it can select one byte out of 2^{16} bytes (64KB)
- ❑ The offset address can be one of the index/pointer registers
- ❑ In 8086, an address is presented always in the form [**segment** : **offset**]

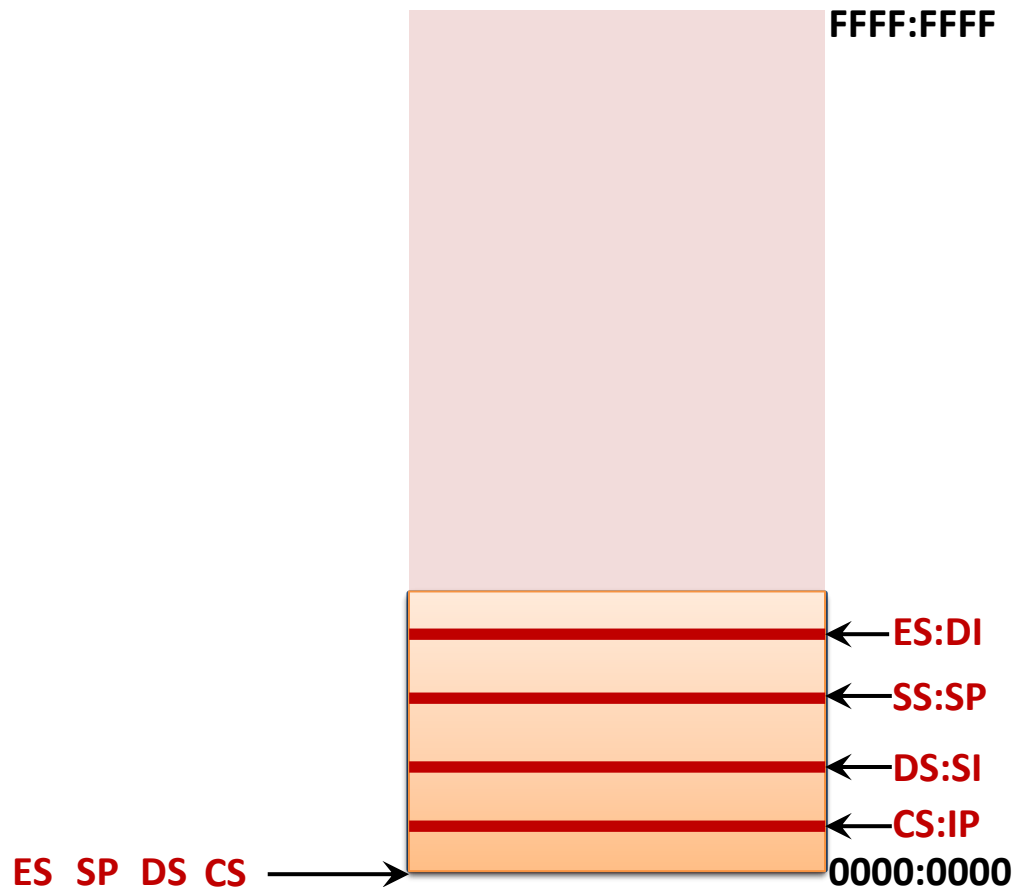
Accessible memory by the 8086

- ❑ At a given moment, the 8086 has access to **four segments** whose addresses are in the segment registers CS, DS, ES and SS.
- ❑ The **code segment** contains the program instructions
- ❑ The **data segment** contains the data manipulated by the program
- ❑ The **stack segment** contains the backup stack
- ❑ The **extra segment ES** may also contain additional data



Rules of 8086 Memory Segmentation

- ❑ The four segments can overlap for small programs
 - ❖ In a minimum system all segments can start at address 0000H
- ❑ The segment can start at any memory address divisible by 16



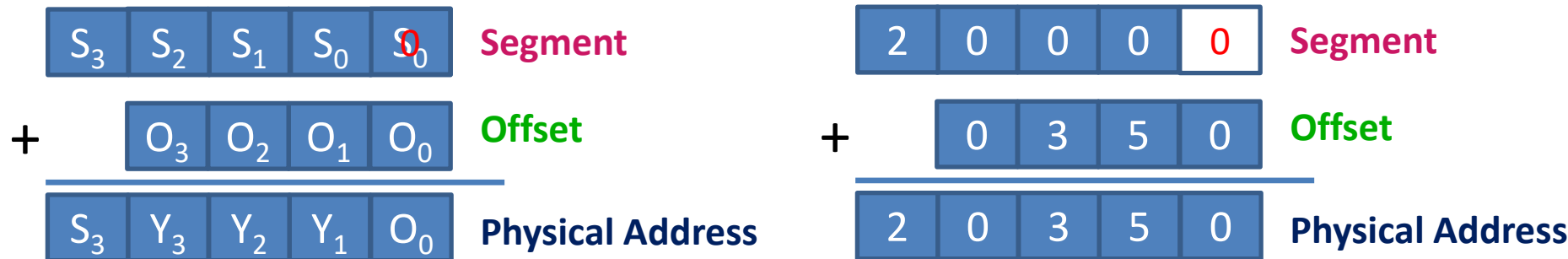
Advantages of Memory Segmentation

- ❑ It allows the addressing of 1MB memory with only 16-bit registers
- ❑ It facilitates the use of separate memory areas of code, data and stack
- ❑ It permits a program or its data to be put in different areas of memory, each time the program is executed
 - ❑ Program can be relocated which is a very useful feature for *multiprogramming*

How does 8086 generate 20-bit address?

❑ Consider that $SR = 2000H$, and $OR = 350H$:

- Left shifting one hex digit of (SR)
- Performs the addition of two registers
- Send the *physical address* on the address bus of 20 bits

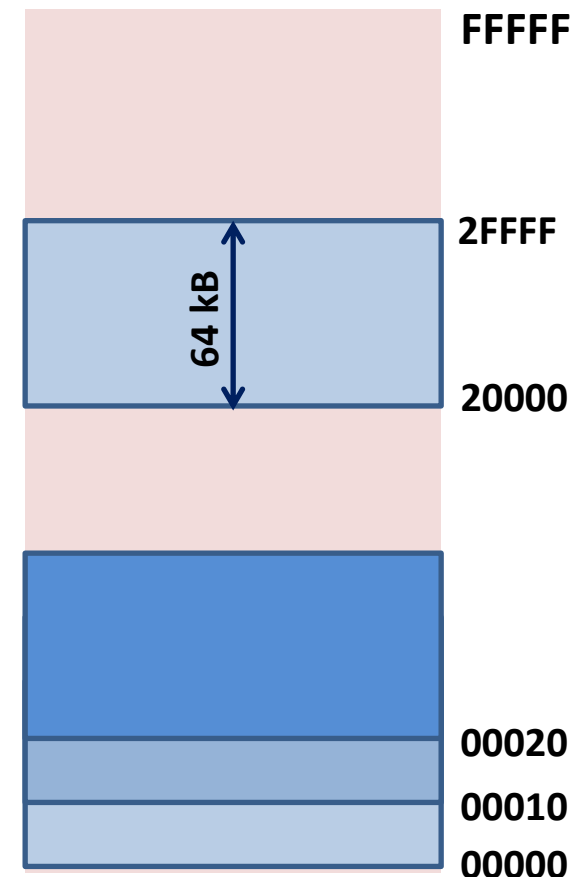
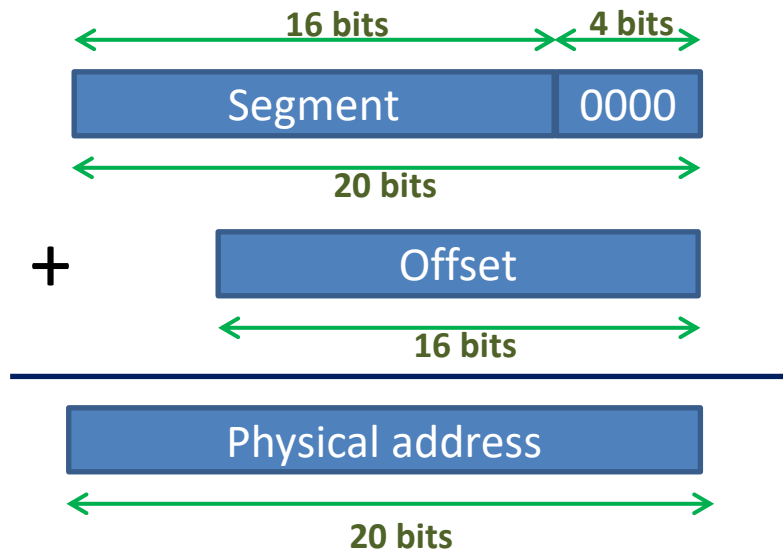


This technique called memory segmentation

How does 8086 generate 20-bit address?

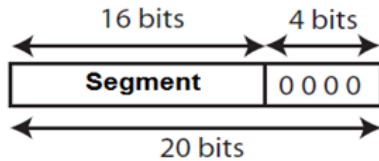
- ❑ An address as a 20-bit quantity (5 hex digits) is called the *physical address* because it is the actually value sent on the **address bus A0 - A19**
- ❑ The *physical address* is calculated by the expression:

$$\text{physical address} = 16 \times \text{segment} + \text{offset}$$



Note : inject 4 zeros $\equiv$ multiplication by $2^4 = 16$

Segment Starts every 16-Bytes



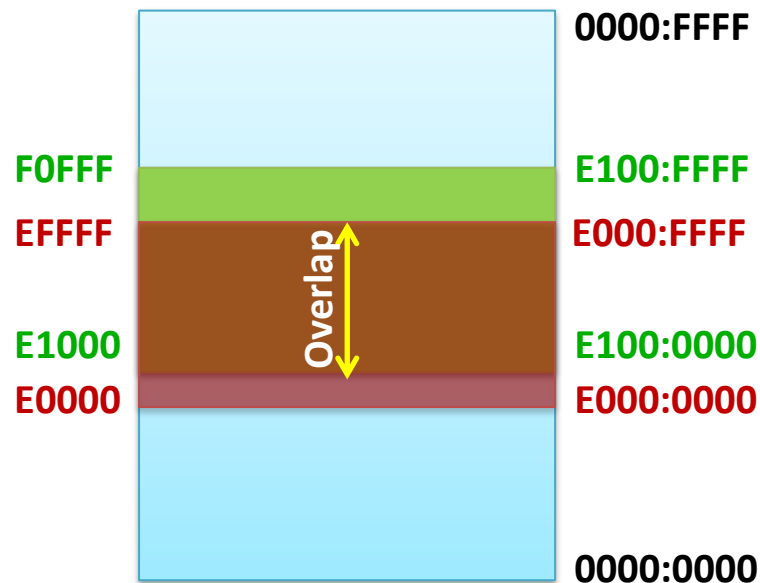
- 1 MB of addressable memory, divided into segments of 64 KB (2^{16})
- A segment starting **every 16 bytes** because of offset 4-bits
- We can place a segment anywhere in the memory **every 16 bytes**, then access at 64 kb space from the segments position

left shifting of 1 hex digit the **Segment register**

Bit : 11-15	Bit : 8-11	Bit : 4-7	Bit : 0-3	Offset 4 bits	Segment starting
0000	0000	0000	0000	0000	0
0000	0000	0000	0001	0000	16
0000	0000	0000	0010	0000	32
0000	0000	0000	0011	0000	48
0000	0000	0000	0100	0000	64
0000	0000	0000	0101	0000	80
0000	0000	0000	1111	0000	240
0000	0000	0001	0000	0000	256
0000	0000	0001	0001	0000	272
0000	0000	0001	1111	0000	496
0000	0000	0010	0000	0000	512
0000	0000	0010	00001	0000	528
0111	1111	1111	1111	0000	1048560
1111	1111	1111	1111	0000	1048575

Overlap of Segments

- ❑ Areas allocated to segments can overlap.
- ❑ *The only rule when selecting a segment is the hex LSD is zero.*
- ❑ So we can begin a segment every 16 bytes



- ❑ We can address the same physical address **00105** with various [**segment** : **offset**] combinations (0010:0005, 0000:0105, 0002:00E5)

$$\begin{array}{r} 0002 \\ + 00E5 \\ \hline 00105 \end{array}$$

$$\begin{array}{r} 0010 \\ + 0005 \\ \hline 00105 \end{array}$$

$$\begin{array}{r} 0000 \\ + 0105 \\ \hline 00105 \end{array}$$

Your Turn (Example)

What physical address corresponds to logical address 028F:0030?

Solution: $028F0 + 0030 = 02920$ (hex)

Always use hexadecimal notation for addresses

What logical address corresponds to the physical address 28F30h?

Many different **segment: offset** (logical) addresses can produce the same physical address 28F30h. Examples:

28F3:0000, 28F2:0010, 28F0:0030, 28B0:0430, . . .

Accessible memory by the 8086

- ❑ The **CS** register is associated with the instruction pointer **IP**, and the next instruction to run is at the logical address **CS: IP**
- ❑ The segment registers **DS** and **ES** may be associated with an index register.
 - Example: **DS:SI**, **ES:DI**.
- ❑ The **DS/ES** segments allow the processor to fetch data stored at an address.
- ❑ The stack segment register may be associated with the pointer registers: **SS: SP** or **SS: BP**

Initial Values of Segment Registers

- ❑ Contents of registers after a *RESET* of the microprocessor:
 - **IP** = 0000H
 - **CS** = FFFFH
 - **DS** = 0000H
 - **ES** = 0000H
 - **SS** = 0000H
- ❑ Since **CS** contains the value FFFFH and **IP** the value 0000H, the first instruction executed by 8086 is at the **logical address FFFFH:0000H**, corresponding to the **physical address FFFF0H** (*bootstrap loader*)
- ❑ This statement is typically a jump to the main program that start up the computer and initializes other segment registers